

Tatami — Phase 3

Composable Kernels: What Has To Line Up

Durwasa Chakraborty

Design document — first draft, nothing decided

1 Purpose and scope

Phase 3 takes its architecture from BOSS: a query engine built out of separable kernels, each free to run on a different device, with a plan split across them at run time rather than compiled for one target. The design note acknowledges that debt and does not restate the architecture, and neither does this document.

What this document does is different, and is the reason it exists. Adopting a kernel decomposition is only half a decision. The other half is the set of things that must be true of *our* generated code for that decomposition to be applicable at all — what a kernel is when the operators are effect handlers rather than library functions, what crosses the boundary between two of them, what makes one legal on a device, and which of the twenty-two decisions already taken in Phase 1 happen to supply those conditions and which happen to obstruct them.

Every proposal below is offered to be argued with. Phase 1 earned the right to write **decided** next to its scenarios by exhausting the alternatives first; nothing here has been through that, and the status column says so.

The unit is the fragment. Throughout, a *fragment* is a region of a query’s lowering that could in principle be dispatched somewhere. The whole question is what delimits one and what may be said about it, so the word is deliberately loose until §3 makes it precise.

2 Status

Ref	Question	Status
K1	What a kernel is	proposed
K2	What crosses the boundary	proposed
K3	The effect signature	proposed
K4	The dispatch predicate	proposed
K5	Composition and transfer placement	proposed
K6	What costing is still for	proposed
K7	The invariant	proposed

Sections §10 and §11 are not questions. They record what Phase 1 already supplies and what it obstructs, and they are the part of this document least likely to change, because they are readings of a settled specification rather than proposals.

3 K1. What a kernel is

BOSS’s kernels are libraries — Velox, ArrayFire — and a plan is split across them by partial evaluation. We have no libraries. Phase 2 leaves us something else: generated accessors that

perform rather than return, and a hole where the handler goes. The design note is explicit that what the handlers do is “deliberately left open here”. This is that question.

Options considered.

- (a) **The kernel is a handler.** A kernel is a handler for some set of effect constructors, together with the fragment running under it. Dispatching a fragment to a different device means installing a different handler over the same code. ← **selected**
- (b) **The kernel is a physical operator**, in the ordinary database sense — a hash join, a scan — and effects are merely how operators request rows from each other.
- (c) **The kernel is a generator pipeline stage**, delimited by where one **next** feeds another.
- (d) **The kernel is a whole relation’s traversal**, so the unit of dispatch is a table.

Proposal. A kernel is a handler plus the fragment it handles. Device selection is handler selection: the fragment’s own code is unchanged, and what changes is the interpretation installed over it.

This is the only option that costs nothing to express, because Phase 2 has already built it. The design note’s §3.2 claims that a polling fragment can be placed “on another core, another node, or a GPU without changing the code that consumes it”, and under (a) that claim is not an aspiration but a restatement of what an effect handler is. Two handlers for **Yield_qty** — one walking a CPU-resident column, one enqueueing a GPU kernel and draining its output — are interchangeable at the point of installation, and the consumer cannot tell which it got.

Rationale. Options (b) and (c) are not wrong so much as premature: they name a unit that we would then have to *relate* to the handler structure Phase 2 generates, and that relation is extra work with no evident payoff. Option (d) is too coarse. A single relation can contain both a fixed-width scalar column that a GPU handles well and a variable-width string column it handles badly, and dispatching the table as one unit forces the worse device on the better column.

Known limitation. Handler boundaries are a property of how the query was written, not of what would be good to dispatch. A consumer that installs one handler around its entire computation presents exactly one kernel, and no decomposition is available at all. This makes the granularity of dispatch depend on the query language of K1’s sibling question — the language must generate handler boundaries where fragments should be, rather than leaving them where a person happened to put them.

4 K2. What crosses the boundary

BOSS composes its kernels over “a unified exchange format for plans and data”. Kernels that cannot agree on a representation cannot compose, so this is prior to everything about dispatch: it is what makes two kernels the same shape of thing.

Proposal. The exchange format is a *column bundle*: for each relation, one dense typed array per column; for each nullable column, a validity mask alongside it; and for a child relation, its **parent_id** and **idx** columns exposed as ordinary arrays like any other.

Nothing in that sentence is invented for Phase 3. Every clause is a reading of a Phase 1 decision, which is the substance of §10.

Consequence. A kernel’s input and output are both column bundles, so the composition of two kernels is defined whenever the second’s expected columns are a subset of the first’s produced ones. No format negotiation, no tagging, and no per-element dispatch.

Known limitation. A column bundle is a materialized representation, and the design note’s §3.2 argues against materialization — a generator’s whole virtue is that its live state is one element and an accumulator. The two are reconciled only if a bundle is understood as the unit of *transfer between kernels* and never as the unit of consumption within one: a kernel streams internally and materializes at its edges. Whether that is achievable, or whether GPU residency forces materialization deeper than the edges, is not settled here and is the most likely place for this proposal to fail.

5 K3. The effect signature

The design note asks whether dispatch can be derived from “the effect signature of the generated accessors”. Before that can be answered, the signature has to exist, and one fact governs how:

OCaml 5 does not track effects in the type system. The PLDI 2021 retrofit shipped handlers without effect types. `Effect.perform` is invisible in a function’s type, and nothing about which effects a fragment performs can be read off a signature the compiler checked. The project’s subtitle, “effect-typed OCaml”, names something we construct, not something we are given.

Options considered.

- (a) **Infer it** — an effect analysis over the generated code, recovering for each fragment the set of constructors it may perform without handling.
- (b) **Generate it** — emit the signature alongside the module, since Phase 1 generated the accessors from the schema and therefore already knows what each one performs. ← **selected**
- (c) **Declare it** — the query language requires the author to annotate, and the annotation is checked.

Proposal. Generate it. The signature is an output of the same pass that emits the accessor, derived from the schema rather than recovered from the code.

Rationale. This is what the design note means by “the provenance of every relation is known: for each file we can determine which code is responsible for populating which part of the database”. Provenance is available because we generated the code; the effect signature is the same fact in a different notation, and recovering by analysis what we could have emitted directly is work performed against ourselves.

The vocabulary. A first cut, stated concretely so that it can be attacked. Phase 2 emits one constructor per field; classify each by what its payload costs a device, not by which field it came from:

Class	Arises from	What it says
$E_{\text{fix}}(\tau)$	int, float, bool	Fixed-width scalar. One array, one stride.
E_{var}	string	Variable width. Needs offsets plus bytes.
$E_{\text{opt}}(E)$	any option column	As E , plus a validity mask.
$E_{\text{ref}}(M)$	a foreign key	A dependent read into M : a gather, not a scan.
E_{rec}	S19’s self-referencing key	A dependent read whose depth is not statically bounded.

A fragment’s signature is the multiset of classes it may perform without handling. Note that the classification is by *layout consequence*: two fields of different names and different meanings but the same width are the same class, because a device cannot tell them apart either.

Known limitation. Generated signatures cover generated code and nothing else. A query that computes anything of its own — an arithmetic predicate, a user-written fold — performs no generated effect, so its signature is empty and the predicate of K4 says nothing useful about it. This is the sharpest constraint on the query language: either it is built only from combinators whose signatures we also generate, or K3 must fall back to option (a) for the parts it does not cover.

6 K4. The dispatch predicate

With K3’s vocabulary the question of §4.1 — which target a fragment “can legally be lowered to” — becomes a predicate on a set rather than an estimate over statistics.

Proposal. A fragment is GPU-legal exactly when its signature contains no E_{rec} , every $E_{\text{ref}}(M)$ it contains targets a relation already resident on the device, and every E_{var} it contains is handled by a kernel that accepts the offsets-plus-bytes encoding. Otherwise it is CPU-only.

Read clause by clause, each says something a device architect would recognise:

- **No E_{rec} .** Recursion is unbounded pointer chasing with data-dependent depth. It is the one class that cannot be given a static iteration count, and it is where GPUs are worst.
- **Resident E_{ref} .** A foreign key is a gather. A gather into device memory is a normal GPU operation; a gather into host memory across the bus, per element, is not. So the clause is not “no joins” but “no joins whose target is elsewhere”, which is a placement question and therefore composes with K5.
- **Encoded E_{var} .** Strings are not excluded, only made conditional on the kernel that handles them. A dictionary-encoded or offsets-encoded string column is workable; a column of independently allocated OCaml strings is not.

Rationale. The predicate is decidable on the signature alone, takes no statistics as input, and returns the same answer for a corpus of ten documents and ten million. That is what distinguishes it from a cost model and is the whole content of the design note’s question.

Known limitation. Decidable is not the same as useful. The predicate says what is *legal*, and legality is a weak claim: a fragment can be GPU-legal and still run worse there, because the predicate has no notion of how many rows will flow through it. That is the honest reading of §4.1’s own word, “legally”, and it means the strong form of the design note’s question — placement *instead of* costing — is not established by this predicate. See K6.

7 K5. Composition and transfer placement

Two kernels compose when the second handles what the first leaves unhandled and their column bundles agree. If they are placed on different devices, data has to move, and where it moves is usually the dominant cost in a heterogeneous plan.

Proposal. A transfer is inserted exactly at a handler boundary where the device changes. There is no other place one can occur.

This is the structural payoff of K1. Because a kernel *is* a handler, and because a device is chosen when a handler is installed, a device change and a handler change are the same event. Transfer points are therefore not discovered by analysis and not estimated: they are enumerable by walking the handler structure of the plan.

Consequence. Two useful things follow. The number of transfers in a plan is a syntactic property of that plan, available before anything is executed or costed. And minimising transfers becomes a question of *where to put handler boundaries*, which is a rewriting problem over a known structure rather than a search over an unknown one.

Known limitation. This makes the plan space smaller than it should be. A transfer that would be profitable in the middle of a long fragment cannot be expressed, because there is no handler boundary there to attach it to, and introducing one means splitting the fragment — which changes what K4 says about each half. Splitting is therefore a plan transformation in its own right, and the interaction between splitting and the predicate is not worked out here.

8 K6. What costing is still for

§4.2 of the design note says plans are enumerated and costed; §6 asks whether dispatch can be derived from effect signatures *rather than* from a cost model. Taken together these are in tension, and the tension has to be resolved before §5’s differential evaluation measures anything interpretable. Three readings, in increasing ambition:

- (1) **Legality only.** The signature filters the candidate targets; the cost model chooses among survivors. $\leftarrow$ **selected**
- (2) **Placement, costing for the rest.** The signature fixes the device outright; costing only picks join implementations and physical operators once it has.
- (3) **Placement instead of costing.** No cost model participates in placement at all.

Proposal. Claim (1), and say so plainly. K4’s predicate decides which targets a fragment may be lowered to; a cost model over the operator repository chooses among them. The contribution is that the legality half is derived structurally, and does not need statistics.

Rationale. (1) is what §4.1 literally says, and it is the only one of the three that K4 as stated actually supports, since the predicate is blind to cardinality. Claiming (3) and then quietly consulting a cost model would make §5’s comparison unfalsifiable — there would be no configuration of the system that could fail to confirm the claim. Claiming (1) and demonstrating it costs less and is worth more.

Known limitation. Claim (1) is a weaker result than §6’s phrasing suggests, and it should be conceded in the write-up rather than blurred. What remains genuinely ours under (1) is still a real claim: that the legality half of dispatch is a compile-time property of generated code, and that a system deriving it this way needs no statistics to know what it *may* do.

Where the statistics would come from, if (2) is ever attempted. S20 of Phase 1 already computes per-path observation counts — present with a value, explicitly null, absent — and declares them diagnostic only. They are selectivity statistics under another name. Promoting them from diagnostic to input would make placement a function of the corpus, which K7 is precisely an attempt to avoid, so this is recorded as an available move and not taken.

9 K7. The invariant

Phase 1 holds together because almost every decision in it defends one property: *the schema is a function of document structure, not of data values*. It is what rules out heuristic map detection, thresholds on optionality, recursion detection, and collision-suffix naming. A phase without such a property is a pile of local choices.

Proposal. The Phase 3 counterpart: *legality is a function of the effect signature, not of the data*. A fragment’s set of admissible targets does not change because the corpus grew.

The two compose into a single statement spanning the project:

The schema is a function of the documents’ structure, and the plan space is a function of the schema.

Neither half mentions data values, so neither does the composition. A corpus that doubles changes no schema, and therefore changes no fragment’s legal targets — only which of them is chosen.

Consequence. This is testable, and cheaply. Run inference and lowering over a corpus, then over a corpus ten times the size with the same structure, and compare the plan spaces. They must be identical. It is a weaker property than §5’s differential correctness oracle and far cheaper to check, which makes it a good early regression test rather than an end-of-project one.

Known limitation. The invariant holds only for the legality half, by construction: under K6’s claim (1) the *chosen* plan does depend on statistics and therefore on data. So the honest statement is that the plan *space* is data-independent while the selection within it is not — which is a sharper and more defensible claim than one about placement as a whole, and is the form that should appear in the write-up.

10 What Phase 1 already supplies

The alignment worth checking is not with BOSS but with our own front end. A kernel decomposition needs a data representation with particular properties, and several Phase 1 decisions — taken for reasons that had nothing to do with devices — turn out to supply exactly those.

Ref	Decided, for its own reasons	What it gives K2
S16, S18	Reject a path whose observed types have no least upper bound.	Every column has exactly one type. A dense typed array with no per-element tag and no branch in the inner loop.
S1a	Widen <code>int</code> to <code>float</code> on mix.	Every column has one width. Uniform stride.
S4, S16b	Nullability is a flag on the column, not a point in the type lattice.	Validity is separable from values: a mask beside a dense array, which is the layout a GPU wants.
S13	An ordering index is required, not optional.	Array order survives as an ordinary integer column, so a sequence is reconstructible from an unordered kernel result.
S14	A scalar array is a table with no module.	The GPU-shaped case arrives with nothing wrapped around it: parent key, index, one value column.
S21.6	<code>parent_id</code> and <code>idx</code> are public record fields.	Join keys are ordinary columns of the bundle, not hidden state a kernel would have to be told about.
S22	Table names are path-qualified and injective.	Every relation has a stable name to be addressed by in a plan.

The one worth dwelling on. S16 and S18 reject heterogeneous paths, and the Phase 1 argument for doing so is entirely about data-dependence: a generated variant would gain a constructor when a document was added. The device consequence was not part of that argument and is nonetheless large. A union column requires a per-element tag and a branch on it, which is close to the worst thing one can put in a GPU kernel; rejecting unions means no column ever needs one. A decision justified on compiler grounds turns out to be a precondition for the systems half, which is the strongest evidence available that the two phases are pulling the same direction.

11 What Phase 1 obstructs

Recorded plainly, because the parts that do not line up are more informative than the parts that do. Several are flagged in `lowering.tex` itself as Phase 3’s problem.

- **S21.5** — `val all : db -> t list`. Materializes an entire table, which is what §3.2 of the design note argues against and what K2 permits only at kernel edges. Phase 1 names `t Seq.t` as the revision and notes it breaks consumers rather than extending them. Under K1 there is a third option Phase 1 could not see: the entry point performs an effect like every other accessor, and the handler decides whether to materialize.
- **S21.2** — **the explicit db handle**. Its own known limitation names option (d), “the store is the effect handler, supplying rows in response to a performed effect”, as the eventual destination. K1 makes that destination the whole design rather than an eventual refinement, so this is the Phase 1 decision Phase 3 most wants revisited.

- **S19 — recursion.** E_{rec} is CPU-only under K4, so a corpus that is mostly trees — comment threads, file systems, organisation charts — gets no GPU benefit at all. This is a real limit on which workloads can demonstrate anything, and it should inform which corpora §5 evaluates over.
- **S1b — out-of-range integers widen to float.** Precision is already lost at ingest, before any kernel runs. If GPU kernels reduce in single precision, that is a second and independent loss on the same column, and §5’s correctness oracle would see the two plans disagree for reasons that have nothing to do with placement. Comparison over such columns needs a tolerance, and saying so in advance is better than discovering it as a failing test.
- **S7 — every nested field costs a join.** Phase 1’s own known limitation observes that flattening “matters for Phase 3 where a flattened column can be reached without one”. Under K3 the difference is exactly E_{fix} against E_{ref} : flattening a total nested object turns a gather into a scan, and therefore turns a residency condition into no condition at all. This is the one place Phase 3 has a concrete reason to ask Phase 1 to reopen a decision.

12 Not addressed

Named so that their absence is deliberate rather than overlooked.

- **The query language.** K3’s limitation constrains it severely — signatures cover generated code only — but nothing here chooses between a combinator library, a PPX quotation, and analysis of ordinary consumer code.
- **Storage.** K2 describes what crosses a kernel boundary and says nothing about what sits behind `db`. Phase 1 declined the question as being about storage rather than lowering; Phase 3 cannot, since a row-major store cannot produce a column bundle without a transposition that would dominate everything measured.
- **Which kernels.** Whether we compose existing ones (ArrayFire, Velox, as BOSS does) or emit our own (OpenCL, as Voodoo does). K4’s predicate is stated over layout consequences and is neutral between them, which is deliberate.
- **The baseline.** §5 compares against “the plan as written”, and that phrase needs a definition sharp enough to implement and honest enough to be fair. A baseline chosen to lose is not evidence.